

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

VaultX

SECURE PASSWORD MANAGER

Submitted By:

RITHISH K - 24MIC0145

M.Tech. Computer Science and Engineering

GURU KARUPPASAMY G – 24MIC0144

M.Tech. Computer Science and Engineering

Submitted to :

Dr. RADHAKRISHNA MOHANTY

Assistant Professor Sr. Grade 1

Department of Quantum AI

School of Computer Science and Engineering

VIT , Vellore

For

CSI3002

APPLIED CRYPTOGRAPHY & NETWORK SECURITY

School of Computer Science and Engineering

October 2026

ABSTRACT

VaultX is a secure password management application developed using Python and Flask to address the challenges of securely storing and managing multiple online credentials. Remembering unique, strong passwords for different websites and services can be difficult, leading users to reuse passwords or store them insecurely. VaultX aims to provide a centralized solution that protects sensitive credential data through cryptographic techniques.

The application incorporates Argon2id for deriving cryptographic keys from the user's master password and AES-256-GCM for encrypting and decrypting stored credentials. Each user's vault is protected using a randomly generated vault key, which is securely wrapped using a key derived from the master password. The system supports multiple users, allowing credentials to be stored and accessed within individual user vaults. In addition, Shamir's Secret Sharing is implemented to divide recovery information into five shares, of which at least three are required to reconstruct the secret. This provides a recovery mechanism for users who need to regain access to their vault. The application also incorporates HMAC-SHA-256-based audit event verification to detect modifications to recorded audit events.

The system is implemented using Flask for the web application, SQLite for database storage, and Python cryptographic libraries for security operations. Functional tests verify key derivation, encryption and decryption, audit event integrity verification, and threshold-based secret recovery. VaultX demonstrates the integration of password-based key derivation, authenticated encryption, secret sharing, and audit integrity verification in a practical password management prototype. Although the application implements several important security mechanisms, further enhancements, including rate limiting, CSRF protection, and production-oriented key and session management, are required before deployment in a real-world environment.

Keywords: Password Manager, Flask, Argon2id, AES-256-GCM, Shamir's Secret Sharing, HMAC-SHA-256, Cryptography, Secure Credential Storage.

1. INTRODUCTION

1.1 Background

The increasing use of online services, web applications, and digital platforms has made password-based authentication an essential part of everyday computing. Users are required to maintain multiple accounts for academic, professional, financial, and personal activities. Managing separate, strong passwords for each account can be difficult, which may lead to password reuse, weak password selection, or insecure storage of credentials.

A password manager provides a centralized mechanism for storing and managing account credentials. However, the password manager itself must protect the stored information against unauthorized access. Cryptographic techniques such as authenticated encryption and password-based key derivation can help protect sensitive data and strengthen the security of stored credentials.

VaultX is a web-based password management application developed using Python and Flask. It uses AES-256-GCM to encrypt stored credentials and Argon2id to derive cryptographic keys from the user's master password. The application also incorporates vault-key protection, threshold-based secret recovery using Shamir's Secret Sharing, and HMAC-SHA-256-based

audit event integrity verification. These mechanisms demonstrate how cryptographic principles can be integrated into a practical application for secure credential management.

1.2 Problem Statement

Storing multiple online credentials securely is a significant challenge for users who access numerous digital services. Maintaining passwords manually can be inconvenient, while storing them in plain-text files or reusing the same password across accounts can expose sensitive information to unauthorized access.

The problem addressed by this project is the development of a password management application that securely stores credentials, separates data between users, and provides a mechanism for recovering access to a protected vault. The system must encrypt sensitive credential information, protect the vault key using a key derived from the master password, and support verification of audit event integrity.

Another challenge is ensuring that recovery and security mechanisms function correctly without compromising the confidentiality of stored credentials. Therefore, the implementation must be tested to verify encryption and decryption, key derivation, recovery operations, and audit event verification.

1.3 Objectives

The major objectives of this project are:

1. To develop a web-based password management application using Python and Flask.
2. To securely store website credentials using AES-256-GCM authenticated encryption.
3. To use Argon2id for deriving cryptographic keys from the user's master password.
4. To protect a randomly generated vault key using cryptographic key wrapping.
5. To maintain separate credential storage for individual users.
6. To implement a recovery mechanism using Shamir's Secret Sharing, with a threshold of three shares out of five.
7. To record audit events and verify their integrity using HMAC-SHA-256.
8. To test the implemented cryptographic operations and recovery mechanisms.
9. To identify implementation limitations and areas for future security improvements.

1.4 Scope

The scope of VaultX is limited to the design, implementation, and functional testing of a software-based password management prototype. The application provides a web interface for user registration, vault access, credential management, recovery, and audit event verification.

The project focuses on applying established cryptographic techniques to protect stored credentials and demonstrate controlled recovery of a vault key. Python is used for the application logic, Flask provides the web framework, and SQLite is used for persistent database storage.

The implementation does not establish that the application is resistant to every possible attack or suitable for production deployment. Additional security measures, including rate limiting, Cross-Site Request Forgery (CSRF) protection, and stronger production-level session and key management, would be required before real-world use. The project is intended primarily as an academic demonstration of cryptographic techniques applied to secure password management.

2. SYSTEM REQUIREMENTS

2.1 Hardware Requirements

VaultX can be developed and executed on a standard personal computer or laptop. No specialized hardware is required because the application uses software-based cryptographic operations and a local database.

- **Processor:** Modern Intel or AMD processor
- **RAM:** 4 GB minimum; 8 GB recommended
- **Storage:** At least 100 MB of free disk space
- **Operating System:** Windows 10/11, Linux, or macOS
- **Network:** Localhost access for development and testing; network connectivity may be required for remote deployment

2.2 Software Requirements

The following software and libraries are used in the implementation:

- **Programming Language:** Python
- **Web Framework:** Flask
- **Database:** SQLite
- **Cryptographic Library:** cryptography
- **Password-Based Key Derivation:** argon2-cffi
- **Secret Sharing Library:** shamir-mnemonic
- **Frontend Technologies:** HTML, CSS, JavaScript, and Jinja2 templates
- **Code Editor:** Visual Studio Code
- **Web Browser:** Google Chrome or another modern browser
- **Terminal:** Windows PowerShell, Command Prompt, or a compatible terminal

2.3 Development Environment

VaultX is developed using Python and the Flask web framework. The application runs locally through Flask's development server and provides a browser-based interface for user registration, vault access, credential management, recovery, and audit verification.

SQLite is used to maintain persistent information about users, encrypted credentials, and audit events. The cryptography library provides AES-GCM encryption and HMAC-based

integrity verification, while argon2-cffi supports Argon2id-based key derivation. The shamir-mnemonic library is used to generate and combine recovery shares.

The application is organized into Python modules for cryptographic operations, recovery functions, database management, and web application logic. HTML templates provide the user interface, and automated test scripts are used to verify important cryptographic and recovery functions.

The project is tested in a local development environment. Its current configuration is intended for academic demonstration and testing rather than direct production deployment.

3. PROPOSED METHODOLOGY AND SYSTEM DESIGN

3.1 Overview

VaultX is a web-based password management system designed to store and manage user credentials using cryptographic protection. The application combines password-based key derivation, authenticated encryption, vault-key protection, secret sharing, and audit event integrity verification.

The system is implemented using Python and Flask, with SQLite serving as the persistent database. The application separates user information, encrypted credentials, and audit records. Sensitive credential data is encrypted before being stored in the database, while cryptographic keys are used to control access to the vault.

The proposed system consists of five main components: user authentication and vault access, credential encryption and management, cryptographic key management, recovery using Shamir's Secret Sharing, and audit event verification.

3.2 System Workflow

The overall workflow of VaultX consists of the following steps:

1. **User registration:** The user provides a username and master password to create a new vault.
2. **Vault-key generation:** The system generates a random vault key for protecting the user's credentials.
3. **Key derivation:** Argon2id derives a cryptographic key from the master password and a randomly generated salt.
4. **Vault-key protection:** The derived key is used to wrap the vault key before it is stored in the database.
5. **Recovery-share generation:** The recovery mechanism generates five shares, with three shares required to reconstruct the recovery secret.
6. **User authentication:** During login, the system derives the key from the supplied master password and attempts to unwrap the stored vault key.
7. **Credential management:** Website names, usernames, and passwords are encrypted and stored in the database. Authorized vault access is required to decrypt and manage them.

8. **Audit event recording:** Relevant operations are recorded in the audit log, with HMAC-SHA-256 signatures used to verify event integrity.
9. **Vault locking:** When the user locks the vault, the application clears the active vault key from its in-memory session mapping and removes the associated session state.
10. **Vault recovery:** The user supplies the required recovery shares and a new master password to recover the secret and update vault-key protection.

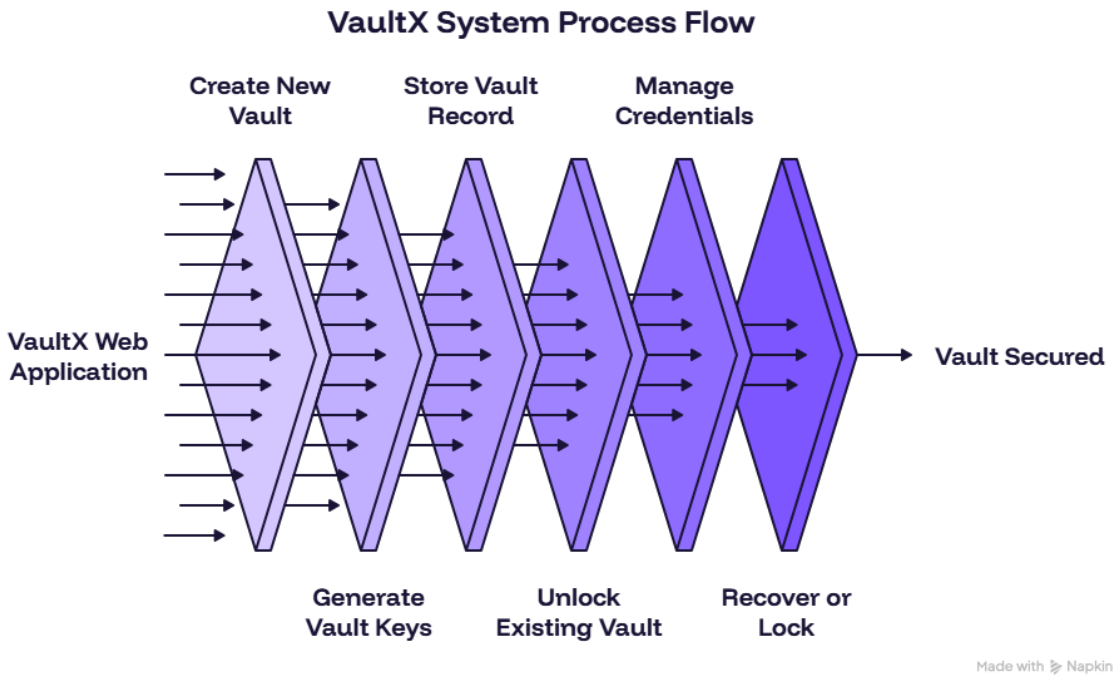


Figure 1:Process Flow of VaultX Secure Password Manager

3.3 System Architecture

The system is organized into four logical layers.

1. Presentation Layer

This layer consists of HTML templates, CSS, JavaScript, and the browser interface. It provides forms and views for registration, login, credential management, recovery, and audit verification.

2. Application Layer

The Flask application handles HTTP requests, user authentication, session management, credential operations, recovery requests, and audit verification. It coordinates communication between the user interface, cryptographic functions, and database.

3. Cryptographic Layer

This layer implements the security-related operations:

- Argon2id for password-based key derivation.
- AES-256-GCM for authenticated encryption of credential data.
- Cryptographic key wrapping and unwrapping.

- Shamir's Secret Sharing for threshold-based recovery.
- HMAC-SHA-256 for audit event integrity verification.

4. Data Storage Layer

SQLite stores user records, encrypted credentials, and audit log entries. User-related records are associated with the corresponding user identifier to support separation of vault data.

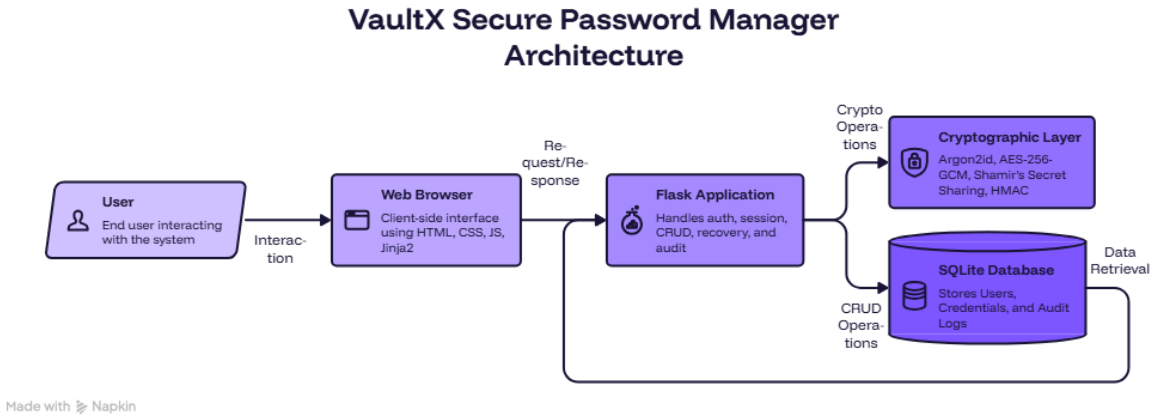


Figure 2: Architecture design of VaultX

3.4 Credential Encryption and Decryption

VaultX uses AES-256-GCM to protect credential information. The credential data is represented as structured data containing fields such as the website, username, and password. This data is encrypted before being stored in the database.

The encryption process uses a 256-bit key and a randomly generated 96-bit nonce. AES-GCM provides confidentiality and authentication, allowing the application to detect unauthorized changes to the encrypted data during decryption.

The general encryption operation is represented as

$$(C, T) = \text{AES-256-GCM}_K(N, P)$$

where:

- (K) represents the encryption key.
- (N) represents the unique nonce used for the encryption operation.
- (P) represents the original credential data.
- (C) represents the encrypted ciphertext.
- (T) represents the authentication tag.

During decryption, the application uses the corresponding key and nonce to recover the original credential data. If authentication fails because the ciphertext or associated authentication information has been modified, decryption is rejected.

The application stores the nonce and encrypted data in the database rather than storing the original credentials in plaintext.

3.5 Cryptographic Key Management

VaultX uses Argon2id to derive cryptographic keys from the user's master password. A randomly generated salt is used during derivation to reduce the effectiveness of precomputed password attacks.

The current implementation uses the following Argon2id parameters:

- Salt length: 16 bytes.
- Derived key length: 32 bytes.
- Time cost: 3 iterations.
- Memory cost: 65,536 KiB.
- Parallelism: 2.

The derived key is used to protect the randomly generated vault key through key wrapping. The wrapped vault key and the required parameters are stored in the database. During authentication, the application derives the key again and attempts to unwrap the vault key. Successful unwrapping allows the application to access the user's encrypted credentials.

This design separates the master password from the key used directly for credential encryption. However, the overall security also depends on password strength, correct nonce handling, secure session management, and protection of key material in memory.

3.6 Recovery Mechanism Using Shamir's Secret Sharing

VaultX incorporates Shamir's Secret Sharing to support recovery of the vault's recovery secret. The implementation generates five mnemonic shares using a threshold configuration in which at least three shares are required for reconstruction.

The recovery process follows these steps:

1. Generate the recovery shares during vault creation.
2. Present the shares to the user for safekeeping.
3. Accept recovery shares when access to the vault must be restored.
4. Validate that the required number of distinct shares has been provided.
5. Reconstruct the recovery secret when the threshold is satisfied.
6. Use the recovered secret to restore the vault key and protect it with a new master password.

The implementation rejects recovery attempts with fewer than three shares and prevents duplicate shares from being counted as separate valid shares.

This mechanism demonstrates threshold-based recovery. Its security depends on the correct handling and storage of recovery shares, and users must keep the shares confidential.

3.7 Audit Event Integrity Verification

VaultX records relevant application events in an audit log. Each event is represented in a canonical JSON format and protected using an HMAC-SHA-256 signature.

HMAC provides a mechanism for checking whether an event has been modified, provided that the verification key remains protected. During audit verification, the application recomputes the HMAC and compares it with the stored signature.

The general operation is represented as:

$$S = \text{HMAC-SHA256}_K(M)$$

where:

- (K) represents the HMAC key.
- (M) represents the canonical audit event.
- (S) represents the resulting authentication code.

If the event data is changed without a corresponding valid signature, verification should fail. This helps detect modifications to recorded audit events. However, HMAC verification alone does not provide a complete tamper-proof audit system, particularly if an attacker can access both the audit records and the relevant verification key.

3.8 Output and Verification

The system provides a browser-based interface for creating a vault, logging in, managing credentials, recovering access, and verifying audit events.

Functional testing is used to evaluate the main cryptographic components. The tests verify that:

- Argon2id produces a derived key of the expected length.
- AES-GCM encryption and decryption recover the original data.
- Modified audit event data fails integrity verification.
- Three valid recovery shares can reconstruct the secret.
- Recovery with fewer than three shares is rejected.

The results are used to assess the functional correctness of the implemented security mechanisms. These tests demonstrate expected behavior under the tested conditions but do not, by themselves, establish resistance to all security attacks.

4. IMPLEMENTATION AND ALGORITHMS

4.1 Implementation Overview

VaultX is implemented using Python and the Flask web framework. The application is organized into separate modules for web request handling, cryptographic operations, database

management, and recovery functionality. This modular structure helps separate application logic from security-related operations and persistent data storage.

The main application is implemented in app.py, which manages user registration, authentication, credential operations, recovery requests, and audit verification. The crypto_utils.py module provides key derivation, authenticated encryption and decryption, vault-key wrapping, and audit event signing and verification. The recovery_utils.py module implements recovery-share generation and reconstruction using Shamir's Secret Sharing. The database.py module initializes and manages the SQLite database schema.

The application uses HTML templates to present the user interface and Python test scripts to verify the behavior of important cryptographic and recovery functions.

4.2 Algorithm 1: User Registration and Vault Creation

Input: Username and master password.

Output: A newly created user vault and recovery shares.

1. Accept the username and master password from the registration form.
2. Validate the username format and minimum master password length.
3. Generate a random salt.
4. Derive a cryptographic key from the master password using Argon2id.
5. Generate a random vault key.
6. Wrap the vault key using the derived key.
7. Store the user record, salt, wrapped vault key, and associated nonce in SQLite.
8. Generate five recovery shares using Shamir's Secret Sharing with a threshold of three.
9. Display the recovery shares to the user.
10. Record the relevant audit event.

The master password is used as input to key derivation; it is not stored directly in the user record.

4.3 Algorithm 2: Credential Encryption and Decryption

Input: Credential data and the vault key.

Output: Encrypted credential data or recovered plaintext credential data.

Encryption procedure:

1. Accept the website name, account username, and password.
2. Serialize the credential fields into structured data.
3. Generate a random 12-byte nonce.
4. Encrypt the serialized data using AES-256-GCM and the vault key.
5. Store the ciphertext, nonce, user identifier, and relevant timestamps in SQLite.

Decryption procedure:

1. Retrieve the selected credential record belonging to the authenticated user.
2. Obtain the ciphertext and corresponding nonce.
3. Decrypt the ciphertext using AES-256-GCM and the vault key.
4. Verify the authentication tag as part of the decryption operation.
5. Return the credential data only if authentication succeeds.

AES-GCM provides authenticated encryption. If the ciphertext or authentication information has been modified, the decryption operation should fail rather than return unauthenticated plaintext.

4.4 Algorithm 3: Master Password-Based Key Derivation

Input: Master password and salt.

Output: A 256-bit derived key.

1. Accept the master password during registration or login.
2. Retrieve the user's stored salt during login.
3. Apply Argon2id using the configured parameters.
4. Generate a 32-byte derived key.
5. During registration, use the derived key to wrap the vault key.
6. During login, use the derived key to attempt to unwrap the stored vault key.
7. Allow access to the vault only when the unwrapping operation succeeds.

The current implementation uses a 16-byte salt, a memory cost of 65,536 KiB, a time cost of 3, and parallelism of 2. These parameters configure the computational and memory cost of deriving the key.

4.5 Algorithm 4: Recovery Using Shamir's Secret Sharing

Input: Recovery shares and a new master password.

Output: Recovered vault-key access and updated key protection.

1. Accept the username, recovery shares, and new master password.
2. Validate the supplied recovery information.
3. Reject the request if fewer than three valid, distinct shares are provided.
4. Use the recovery library to reconstruct the secret from the supplied shares.
5. Use the reconstructed secret according to the application's recovery implementation to restore vault-key access.
6. Generate a new salt for the new master password.
7. Derive a new wrapping key using Argon2id.
8. Wrap the vault key using the new derived key.
9. Update the stored key-wrapping information in SQLite.

10. Record the recovery event and establish the recovered vault session.

This algorithm uses a threshold of three shares out of five. The recovery process is intended to allow access to be restored without requiring the original master password.

4.6 Algorithm 5: Audit Event Integrity Verification

Input: Stored audit event and its HMAC signature.

Output: Integrity verification result.

1. Retrieve the relevant audit events for the authenticated user.
2. Read the canonical JSON representation of each event.
3. Recompute the HMAC-SHA-256 value using the verification key.
4. Compare the computed value with the stored signature using the implemented verification procedure.
5. Mark the event as valid if the signature matches.
6. Report a verification failure if the signature does not match.

HMAC-SHA-256 allows the application to detect modifications to audit event data when the verification key remains protected. It does not, by itself, prevent deletion of audit records or protect against an attacker who can also compromise the verification key.

4.7 Implementation Notes

The implementation uses separate Python modules to improve code organization and maintainability. Flask handles HTTP requests and coordinates the operations, while the cryptographic utilities perform key derivation, encryption, decryption, and integrity verification. SQLite provides persistent storage for user information, credentials, and audit events.

The vault key is maintained in application memory while a vault is unlocked. Locking the vault clears the corresponding in-memory key mapping and session state. This design supports the application's local workflow, although production deployment would require further hardening of session handling, secret management, request protection, and authentication controls.

The test scripts test_crypto.py and test_recovery.py are used to verify key cryptographic operations, audit integrity detection, and threshold-based recovery. Successful tests demonstrate functional correctness for the tested cases, but they do not constitute a comprehensive security audit.

5. IMPLEMENTATION

5.1 User Interface Implementation

The user interface of VaultX is developed using HTML, CSS, JavaScript, and Jinja2 templates. Flask renders the appropriate interface according to the current application state. The interface provides separate views for vault creation, user login, credential management, recovery, and audit verification.

The registration interface collects the username and master password required to create a vault. The login interface allows registered users to unlock their vaults. The credential management interface provides operations for adding, viewing, searching, and deleting stored credentials. Additional interfaces support recovery-share submission and audit event verification.

The frontend communicates with the Flask application through HTTP requests and HTML forms. Server-side validation and cryptographic operations are performed by the backend rather than relying exclusively on client-side validation.

5.2 Flask Application Implementation

The main application logic is implemented in app.py using Flask routes. Each route handles a specific operation and coordinates the required database and cryptographic functions.

The principal application routes are:

Route	Implementation purpose
/	Displays the vault creation interface and processes registration requests
/vault	Handles login when locked and credential management when unlocked
/audit	Verifies the integrity of the user's audit events
/lock	Records the lock operation and clears active vault session state
/recover	Processes recovery shares and updates vault-key protection

The application validates user input, retrieves the appropriate database records, and invokes the required cryptographic functions. Credential operations are restricted using the authenticated user's identifier to maintain separation between user vaults.

5.3 Cryptographic Utility Implementation

The file crypto_utils.py contains the cryptographic functions used by the application. Separating these functions from the Flask routes improves code organization and makes individual operations easier to test.

The module implements the following operations:

- **Salt generation:** Creates a random salt for password-based key derivation.
- **Key derivation:** Uses Argon2id to derive a 256-bit key from the master password and salt.
- **Credential encryption:** Encrypts credential data using AES-256-GCM with a randomly generated nonce.
- **Credential decryption:** Decrypts the stored ciphertext and verifies its authentication tag.
- **Vault-key wrapping:** Protects the vault key using the derived key.
- **Vault-key unwrapping:** Recovers the vault key after successful authentication.

- **Audit signing and verification:** Uses HMAC-SHA-256 to generate and verify audit event authentication codes.

These functions form the main cryptographic layer of VaultX and are called by the Flask application when required.

5.4 Database Implementation

VaultX uses SQLite for persistent storage. Database initialization and schema management are implemented in database.py. The database contains three principal tables: users, credentials, and audit_log.

Table	Principal stored information
users	User identifier, username, salt, wrapped vault key, wrapping nonce, and creation timestamp
credentials	Credential identifier, user identifier, encrypted credential data, nonce, and timestamps
audit_log	Audit event identifier, user identifier, event JSON, and HMAC signature

The username field is configured to enforce case-insensitive uniqueness. The credentials and audit log tables associate records with a user identifier through foreign-key relationships.

Sensitive credential data is stored in encrypted form. The database stores the wrapped vault key rather than the raw vault key. This arrangement separates persistent data storage from the active key used by the application to access a vault.

5.5 Recovery Module Implementation

The file recovery_utils.py implements the recovery mechanism using the shamir-mnemonic library. The configured scheme generates five recovery shares and requires at least three shares for reconstruction.

During recovery, the application receives the supplied shares and validates the recovery requirements. If the required threshold is not met or duplicate shares are provided, the recovery operation is rejected. When sufficient valid shares are available, the library combines them to reconstruct the secret.

The recovery functionality is integrated with the Flask application through the /recover route. The application uses the recovered secret according to its implemented recovery procedure and updates the vault-key protection using the new master password.

5.6 Session and Vault Locking Implementation

VaultX maintains unlocked vault keys in an in-memory dictionary named UNLOCKED_VAULTS, associated with session-specific vault identifiers. This allows the application to use the appropriate vault key for credential operations while a vault is unlocked.

When the user locks the vault, the application records the lock event, removes the active vault key from the in-memory mapping, and clears the associated session state. Subsequent credential operations require the vault to be unlocked again.

Because the unlocked keys are maintained in process memory, restarting the application clears this in-memory state. Production deployment would require additional review of session expiration, concurrent sessions, secret handling, and process-level security.

5.7 Audit Log Implementation

The audit mechanism records relevant application events in the SQLite audit_log table. Each event is represented as canonical JSON and accompanied by an HMAC-SHA-256 signature.

When the user requests audit verification, the application retrieves the user's audit events and checks their signatures using the implemented verification function. A matching signature indicates that the event data is consistent with the signed version, assuming the verification key remains protected.

The audit mechanism provides a means of detecting modifications to recorded event data. It should not be interpreted as a complete tamper-proof logging system because it does not independently prevent deletion of records or protect against compromise of the verification key.

5.8 Testing Implementation

VaultX includes two test scripts, test_crypto.py and test_recovery.py, for checking the principal cryptographic and recovery functions.

The cryptographic tests cover Argon2id key derivation, AES-GCM encryption and decryption, successful audit signature verification, and detection of modified audit events. The recovery tests verify successful reconstruction with the required number of shares and rejection of attempts using fewer than three shares.

These tests help confirm that the implemented functions behave as expected for the selected test cases. Additional integration, security, and deployment testing would be required before the application could be considered suitable for production use.

6. APPLICATION WORKING

6.1 Overview of Application Working

VaultX is a web-based password management application developed using Python, Flask, and SQLite. It provides a centralized interface for creating a password vault, authenticating users, managing stored credentials, recovering vault access, and verifying audit event integrity.

The application follows a request-response model. Users interact with the web interface through a browser, while Flask processes the submitted requests and invokes the appropriate database and cryptographic functions. Sensitive credential information is encrypted before being stored in the database, and access to the vault requires successful authentication.

The main application workflow includes vault creation, user login, credential management, recovery-share generation, vault recovery, audit verification, and vault locking.

6.2 Vault Creation

The vault creation process allows a new user to register with a unique username and a master password. The application validates the submitted information before creating the user's vault.

During registration, the application generates a random salt and derives a cryptographic key from the master password using Argon2id. A separate random vault key is generated to protect the user's credentials. The derived key is used to wrap the vault key, and the resulting information is stored in the SQLite database.

The application also generates five recovery shares using Shamir's Secret Sharing. A minimum of three shares is required to reconstruct the recovery secret. These shares provide an additional recovery mechanism if the user needs to restore access to the vault.

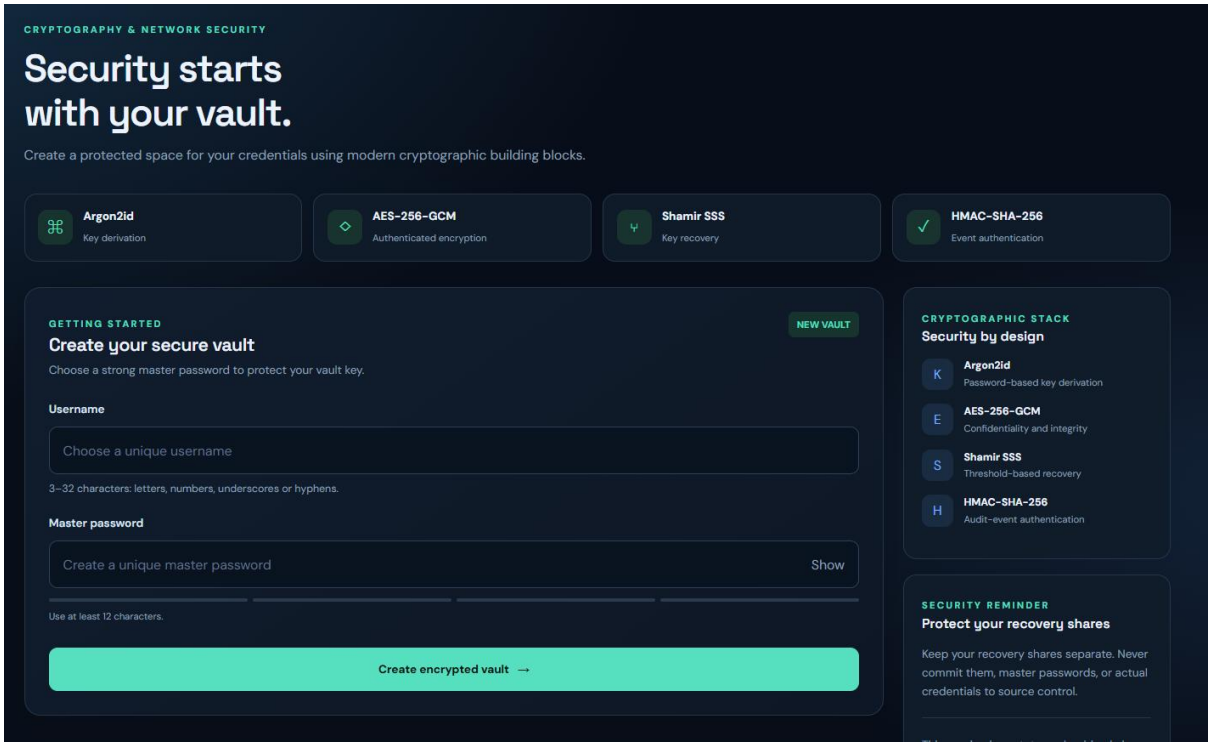


Figure 3:Vault creation Interface

6.3 User Authentication and Vault Unlocking

When an existing user attempts to log in, the application retrieves the corresponding user record from the database. The submitted master password and stored salt are used to derive the cryptographic key through Argon2id.

The derived key is then used to unwrap the stored vault key. If the operation succeeds, the vault key becomes available to the application for the user's credential operations. If authentication fails, the application displays an error message and does not grant access to the vault. This mechanism avoids storing the master password directly in the database and separates password-based key derivation from the key used to encrypt credentials.

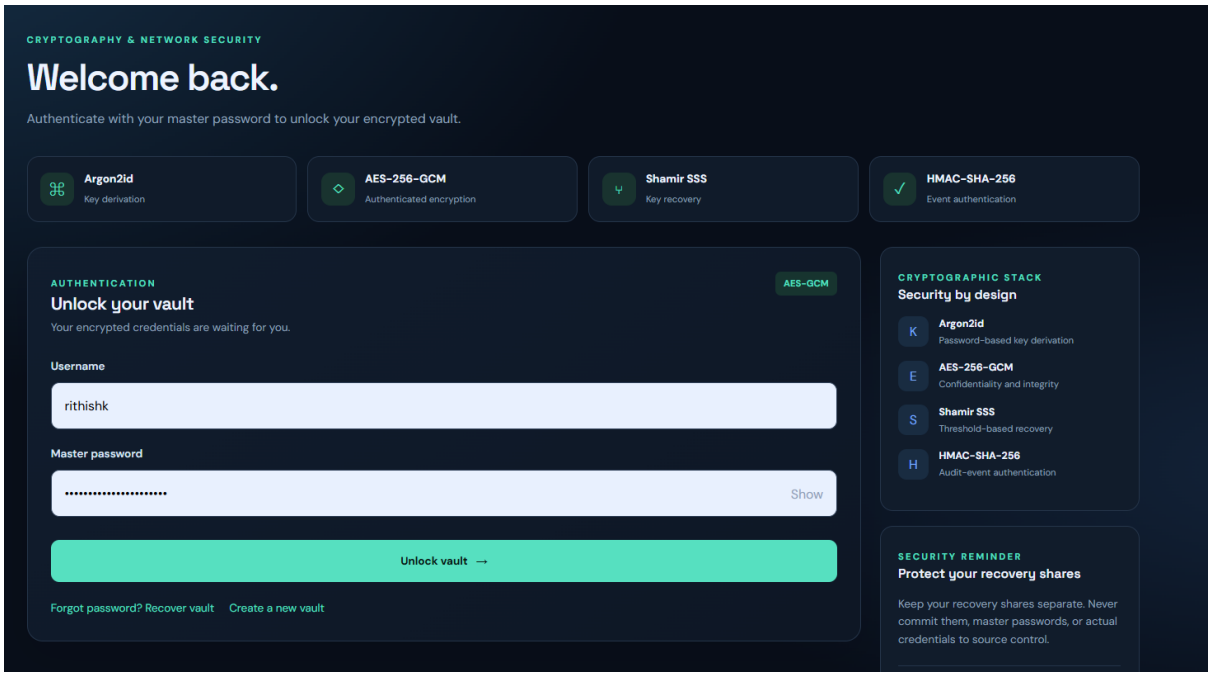


Figure 4:User Authentication Interface

6.4 Credential Management

After successfully unlocking the vault, the user can manage stored website credentials through the application interface. The main credential operations include adding, viewing, searching, and deleting credential records, depending on the available interface functionality.

When a new credential is added, the application collects information such as the website name, account username, and password. The credential data is serialized and encrypted using AES-256-GCM with the vault key and a randomly generated nonce. The ciphertext and associated information are then stored in SQLite.

When the user requests a stored credential, the application retrieves the corresponding encrypted record and decrypts it using the vault key. AES-GCM verifies the authenticity of the encrypted data during decryption. If authentication fails, the operation is rejected.

Credential operations are associated with the authenticated user's identifier to help prevent one user from accessing another user's records.

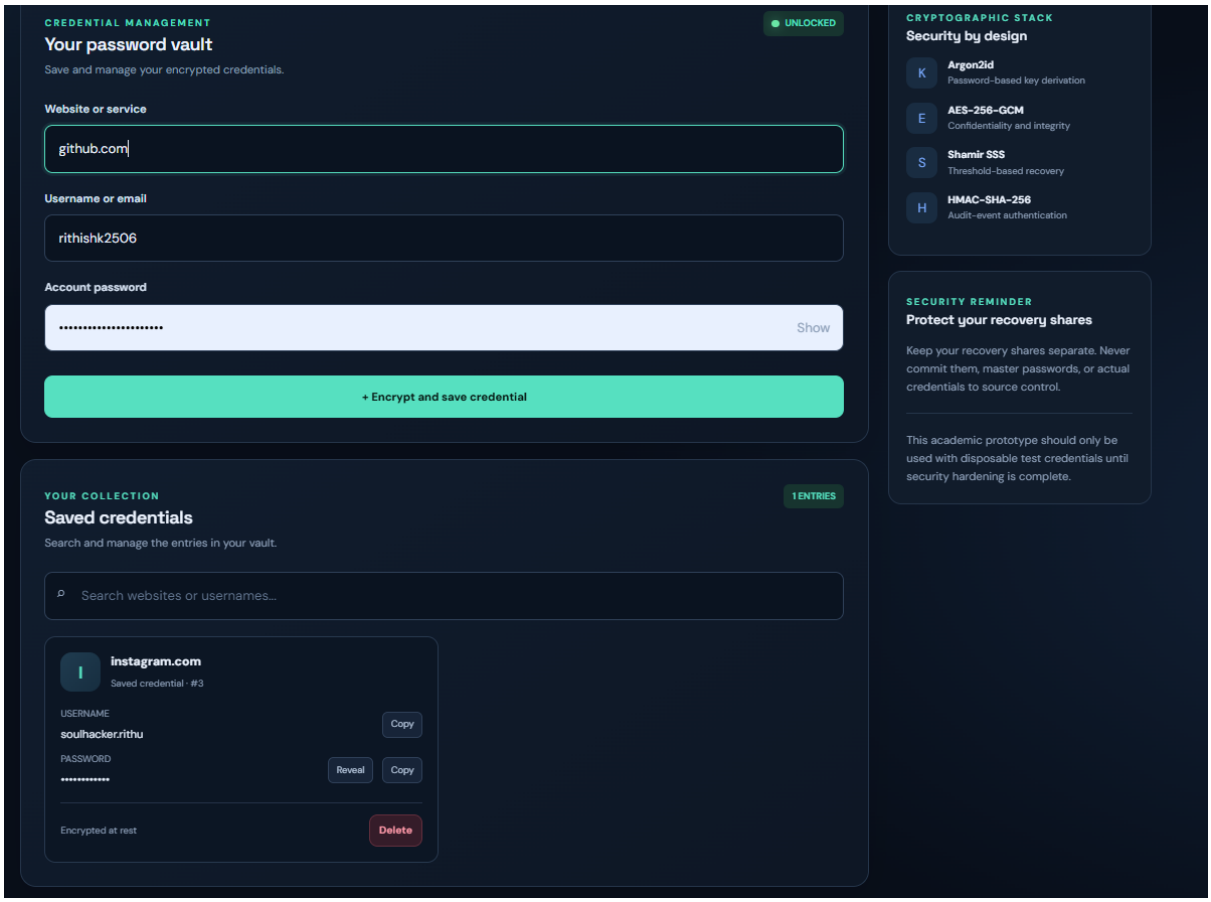


Figure 5:Credential Management Dashboard

6.5 Recovery Share Generation

VaultX provides a threshold-based recovery mechanism using Shamir's Secret Sharing. During vault creation, five recovery shares are generated according to a three-out-of-five threshold configuration.

The user is required to preserve these shares securely. The recovery mechanism permits reconstruction of the secret when the required number of valid shares is supplied. Fewer than three shares are insufficient for recovery under the configured threshold.

The use of recovery shares provides an alternative recovery mechanism without requiring the original master password. However, the recovery shares must be protected against unauthorized access because possession of sufficient valid shares may enable recovery.

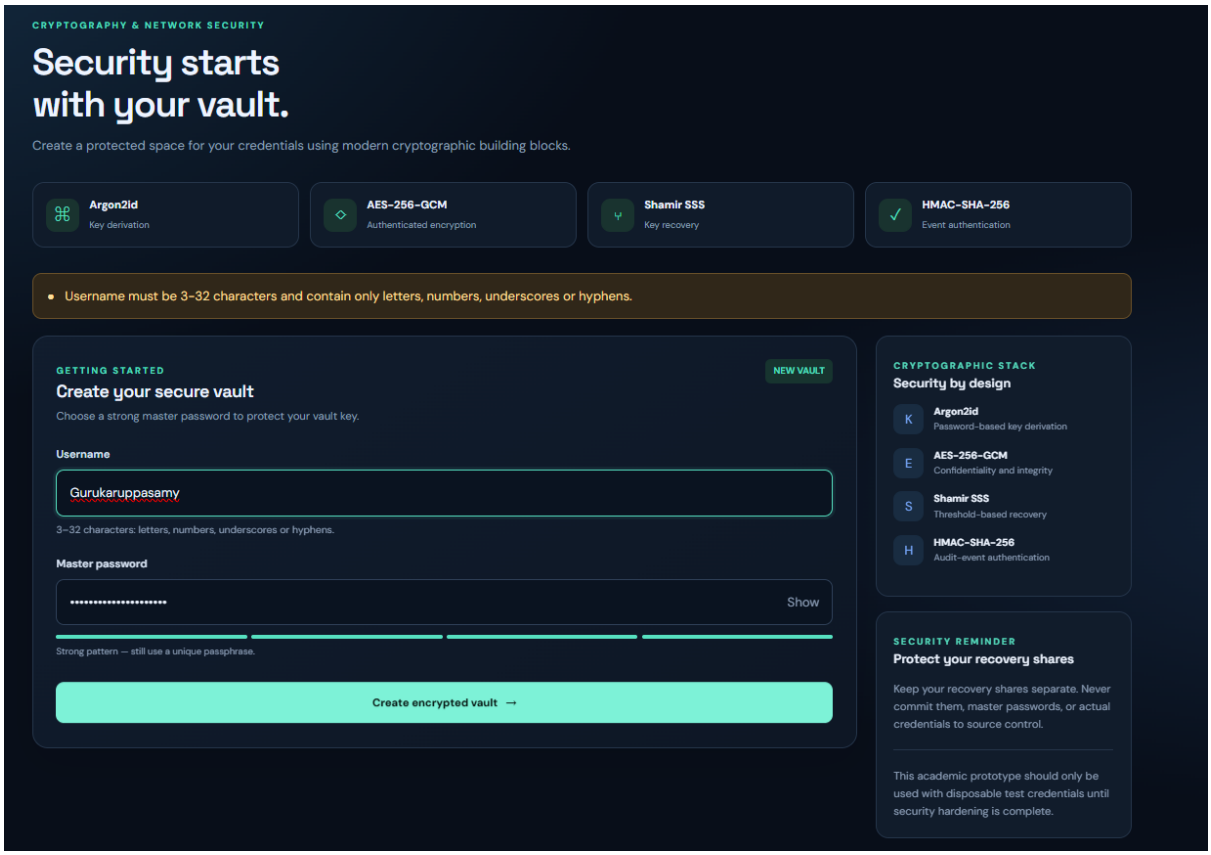


Figure 6:New Vault creation

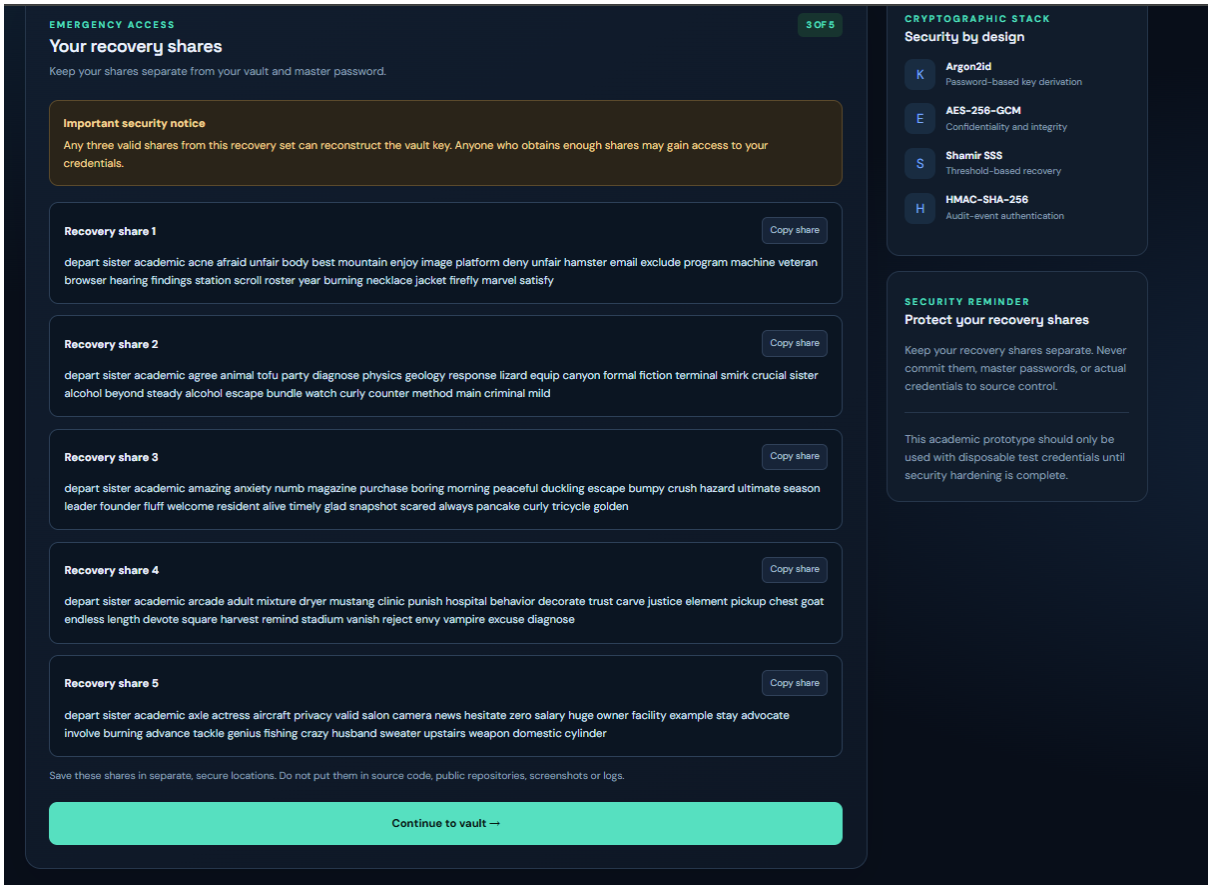


Figure 7:Recovery Shares Interface

6.6 Vault Recovery

The recovery process allows a user to restore access to a vault by providing the required recovery information and a new master password.

The application accepts the username, recovery shares, and new password. It validates the supplied recovery information and rejects attempts that do not meet the required threshold. When sufficient valid shares are provided, the recovery library reconstructs the secret.

The application then uses the recovered secret according to its implemented recovery procedure and updates the vault-key protection using a key derived from the new master password. The updated key-wrapping information is stored in the database.

This process demonstrates threshold-based secret recovery and password replacement. Recovery depends on the availability of sufficient valid shares and the correct execution of the application's recovery logic.

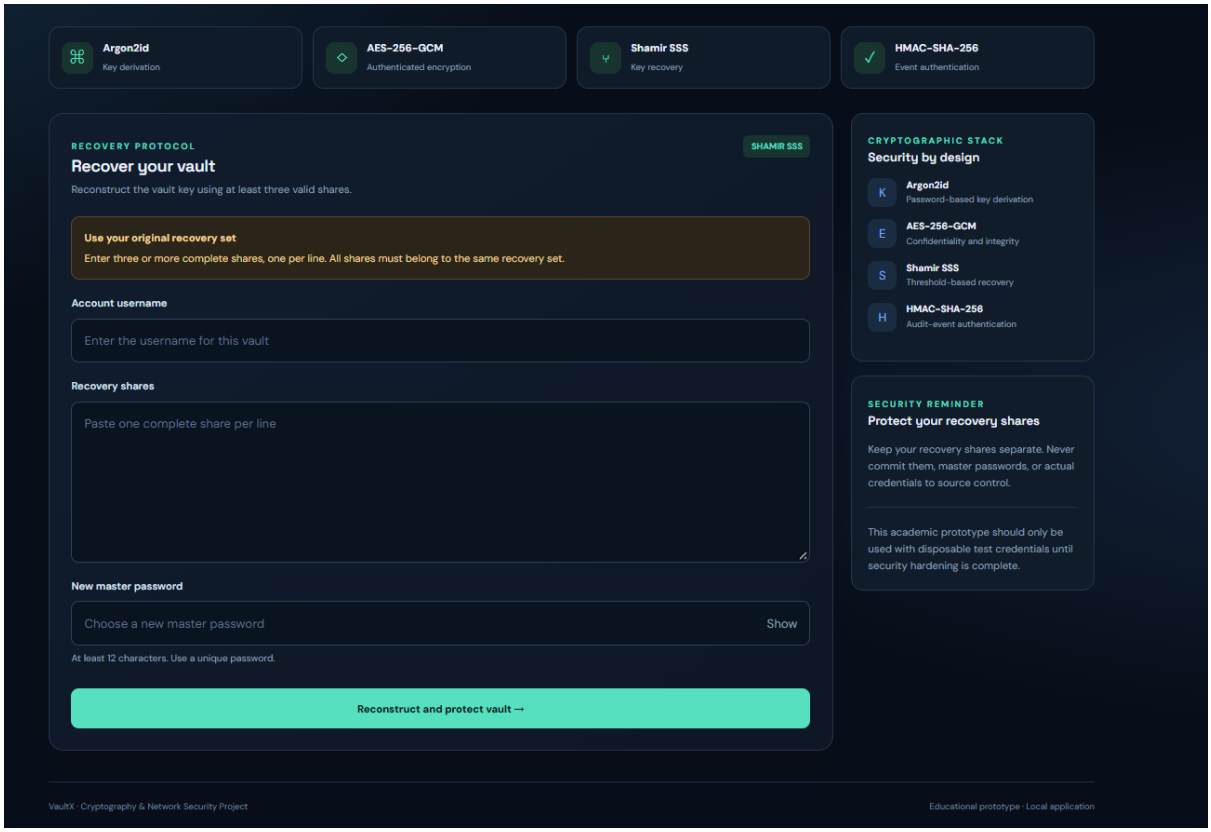


Figure 8:Vault Recovery Interface

6.7 Audit Event Verification

VaultX maintains an audit log containing selected application events. Each audit event is represented in canonical JSON format and accompanied by an HMAC-SHA-256 signature.

When the user accesses the audit verification functionality, the application retrieves the relevant audit records and recomputes the HMAC values using the implemented verification procedure. The computed values are compared with the stored signatures to determine whether the event data remains consistent with the signed version.

If an event is modified without generating a valid replacement signature, the verification process should detect the modification. This mechanism helps identify changes to audit event data, provided the verification key remains protected.

HMAC-based verification does not independently prevent deletion of audit records or protect against compromise of the verification key.

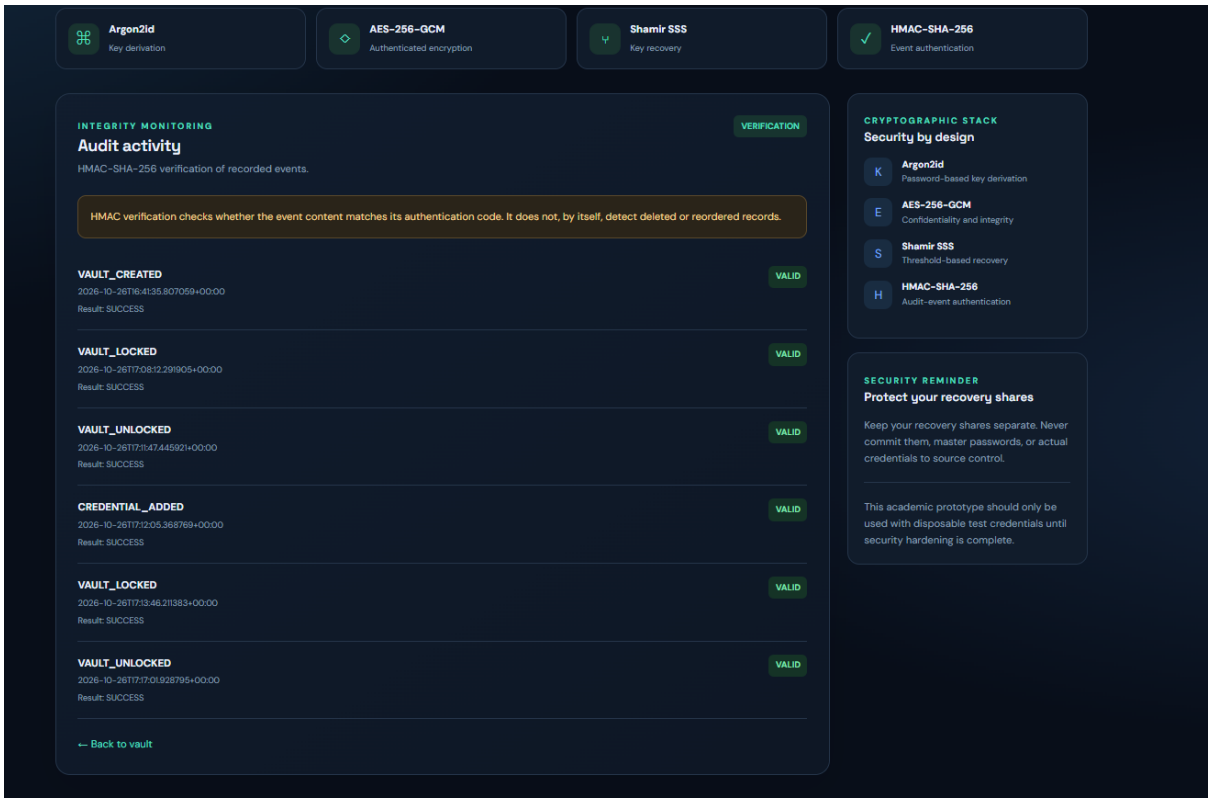


Figure 9: Audit logs interface

6.8 Vault Locking

VaultX provides a locking mechanism to terminate active access to an unlocked vault. When the user requests the lock operation, the application records the relevant event, removes the associated vault key from the in-memory mapping, and clears the corresponding session state.

After the vault is locked, the user must authenticate again before performing further credential operations. The in-memory key handling helps separate active vault access from the persistent database records.

Since unlocked vault keys are maintained in application memory, restarting the Flask process clears the in-memory state. Further safeguards would be required for production deployment, including improved session expiration, request protection, and secure management of application secrets.

6.9 Application Modules

The application is divided into separate modules to improve organization and maintainability.

Module or file	Purpose
app.py	Handles Flask routes, registration, authentication, credential operations, recovery, and audit verification
crypto_utils.py	Implements Argon2id key derivation, AES-GCM encryption and decryption, vault-key wrapping, and HMAC operations
recovery_utils.py	Generates and reconstructs recovery shares using Shamir's Secret Sharing
database.py	Initializes and manages the SQLite database schema

test_crypto.py	Tests key derivation, encryption, decryption, and audit event integrity
test_recovery.py	Tests threshold-based recovery and rejection of insufficient shares
templates/index.html	Provides the browser interface using HTML, CSS, JavaScript, and Jinja2

6.10 Application Security Considerations

VaultX demonstrates several security mechanisms, including authenticated encryption, password-based key derivation, cryptographic key wrapping, threshold-based recovery, and audit event integrity verification.

However, the application is an academic prototype rather than a production-ready password manager. Its security depends on the correct implementation and configuration of the cryptographic functions, the protection of master passwords and recovery shares, and the security of the application environment.

Additional measures, including rate limiting, CSRF protection, secure configuration of the Flask secret key, improved session and key management, and broader security testing, should be considered before deployment in a real-world environment.

7. EXPERIMENTAL RESULTS AND ANALYSIS

7.1 Overview

The experimental evaluation of VaultX was carried out to verify the correctness of its cryptographic operations, secure credential storage, recovery mechanism, and audit event integrity. The application was developed using Python and Flask, with SQLite serving as the database. The experiments focused on validating the implemented security mechanisms and confirming that the application performs the intended operations.

7.2 Cryptographic Function Testing

The cryptographic utility module was tested to verify key derivation, encryption, decryption, and audit event integrity. Argon2id was used to derive a cryptographic key from the master password, while AES-256-GCM was used to encrypt and decrypt credential data.

The tests checked whether encrypted data could be decrypted correctly using the appropriate key and whether modifications to authenticated audit events could be detected. These tests were intended to verify the functional correctness of the implemented cryptographic operations.

7.3 Recovery Mechanism Testing

The recovery mechanism was evaluated using Shamir's Secret Sharing. The implementation generates five recovery shares and requires at least three valid shares to reconstruct the secret.

The test procedure verified that the secret could be recovered using the required threshold of shares and that recovery was rejected when fewer than three shares were supplied. This evaluation demonstrates the intended threshold-based recovery behavior of the implemented module.

7.4 Application Functionality Testing

The application's main functions were checked through its web interface. The evaluation covered user registration, authentication, credential management, vault locking, recovery, and audit event verification.

The application was also checked for separation of credentials between users and for the handling of authentication failures. Only behaviors confirmed through actual execution should be reported as successfully tested.

7.5 Summary of Experimental Evaluation

The experimental evaluation provides a functional assessment of the security mechanisms implemented in VaultX. The cryptographic tests examine encryption and decryption correctness, key derivation, and audit integrity, while the recovery tests examine threshold-based secret reconstruction.

These results support the functional validity of the implemented prototype. However, passing functional tests does not establish resistance to every possible attack. Further security testing, including rate-limit testing, CSRF testing, session security evaluation, and penetration testing, would be required before considering deployment in a production environment.

8. CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

VaultX is a secure password manager prototype developed using Python, Flask, and SQLite to demonstrate the application of cryptographic techniques in password management. The system integrates Argon2id-based key derivation, AES-256-GCM authenticated encryption, cryptographic key wrapping, Shamir's Secret Sharing, and HMAC-SHA-256-based audit event verification.

The application provides user registration and authentication, encrypted credential management, threshold-based vault recovery, audit event verification, and vault locking. The implemented cryptographic and recovery modules were tested to evaluate their functional correctness. The results demonstrate how established cryptographic mechanisms can be integrated into a web application to protect stored credentials and support recovery operations.

The project also highlights the importance of secure key management, authenticated encryption, access control, and integrity verification in password management systems. Although the prototype demonstrates these concepts, additional security hardening and comprehensive testing are necessary before it can be used to store real-world credentials. Therefore, VaultX serves as an academic implementation of cryptographic security principles rather than a production-ready password manager.

8.2 Future Enhancements

Several improvements can be introduced to strengthen VaultX and extend its functionality.

1. **Multi-Factor Authentication:** Introduce an additional authentication factor to provide stronger protection against compromised master passwords.

2. **Rate Limiting and Account Protection:** Implement rate limiting, login attempt monitoring, and temporary account lockout mechanisms to reduce the risk of repeated password-guessing attempts.
3. **CSRF Protection and Input Validation:** Add CSRF tokens to state-changing web requests and strengthen validation of all user-supplied inputs.
4. **Improved Session and Key Management:** Use a securely configured Flask secret key, strengthen session handling, and improve the protection and lifecycle management of sensitive keys in memory.
5. **Password Strength Analysis:** Integrate password strength assessment and warnings for weak or potentially reused passwords.
6. **Secure Password Generation:** Provide a configurable password generator that creates strong, random passwords for different websites and services.
7. **Automated Security Testing:** Conduct additional tests covering authentication failures, unauthorized access, database isolation, session handling, and common web application vulnerabilities.
8. **Secure Deployment:** Configure HTTPS, production-grade server settings, secure cookie attributes, monitoring, and appropriate backup procedures before any real-world deployment.

These enhancements would improve the security, usability, and reliability of VaultX while providing opportunities for further study of applied cryptography and secure web application development.

REFERENCES

1. B. Kaliski, “RFC 2898: PKCS #5: Password-Based Cryptography Specification Version 2.0,” Internet Engineering Task Force (IETF), 2000.
2. National Institute of Standards and Technology (NIST), *Advanced Encryption Standard (AES)*, Federal Information Processing Standards Publication 197, 2001.
3. A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” RFC 9106, Internet Engineering Task Force (IETF), 2021.
4. National Institute of Standards and Technology (NIST), *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, Special Publication 800-38D, 2007.
5. A. Shamir, “How to Share a Secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
6. H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” RFC 2104, Internet Engineering Task Force (IETF), 1997.
7. Pallets Projects, “Flask Documentation.” Available: https://flask.palletsprojects.com/
8. Python Software Foundation, “Python Documentation.” Available: https://docs.python.org/3/

9. SQLite Consortium, “SQLite Documentation.” Available: https://www.sqlite.org/docs.html
10. The Python Cryptographic Authority, “Cryptography Documentation.” Available: https://cryptography.io/
11. Hynek Schlawack and contributors, “argon2-cffi Documentation.” Available: https://argon2-cffi.readthedocs.io/
12. Trezor, “shamir-mnemonic: Python Implementation of SLIP-0039.” Available: https://github.com/trezor/python-shamir-mnemonic

Appendices are included in separate file.